

AI_ASSIST CODING LAB ASSIGNMENT -16.2

Program Name: B. Tech

Academic Year: 2025-2026

Course Code: 23CS002PC304

Course Title: AI Assisted Coding

Year/Sem: III / II

Student Name: Sanjay Karupothula

Roll No: 2303A52337

Batch: 34

Assignment Number: 16.2 / 24

Lab 16 – Database Design and Queries: Schema Design and SQL Generation

Lab Objectives

- Understand schema design principles for relational databases.
 - Use AI-assisted tools to generate SQL schema definitions.
 - Write and refine SQL queries (CRUD + aggregate operations).
 - Explore how AI can optimize queries and ensure normalization.
-

Task 1 – Design Database Schema for Student Management System

Prompt Used

Generate SQL CREATE TABLE statements for Students, Courses and Enrollments tables with primary keys and foreign keys following normalization rules.

SQL Code

```
CREATE TABLE Students (  
  student_id INT PRIMARY KEY,  
  name VARCHAR(100),  
  email VARCHAR(100)  
);
```

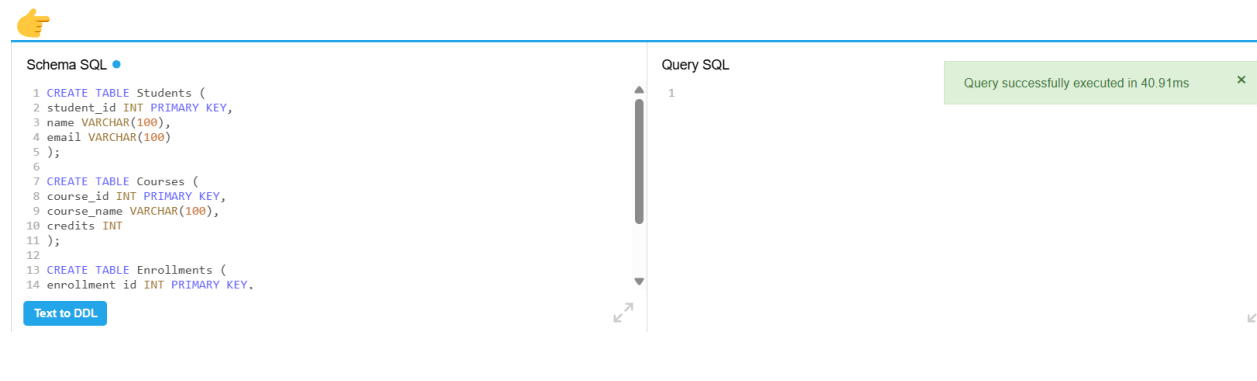
```
CREATE TABLE Courses (  
  course_id INT PRIMARY KEY,  
  course_name VARCHAR(100),  
  credits INT  
);
```

```
CREATE TABLE Enrollments (  
  enrollment_id INT PRIMARY KEY,  
  student_id INT,  
  course_id INT,  
  FOREIGN KEY (student_id) REFERENCES Students(student_id),  
  FOREIGN KEY (course_id) REFERENCES Courses(course_id)  
);
```

Output

Tables Students, Courses and Enrollments created successfully.

Screenshot of Execution – Task 1



Task 2 – Insert Sample Records

Prompt Used

Generate INSERT SQL statements to populate Students, Courses and Enrollments tables.

SQL Code

```
INSERT INTO Students VALUES
(1, 'Arjun', 'arjun@gmail.com'),
(2, 'Meera', 'meera@gmail.com'),
(3, 'Rahul', 'rahul@gmail.com');
```

```
INSERT INTO Courses VALUES
(101, 'Database Systems', 4),
(102, 'Data Structures', 3),
(103, 'Operating Systems', 4),
(104, 'AI Fundamentals', 3);
```

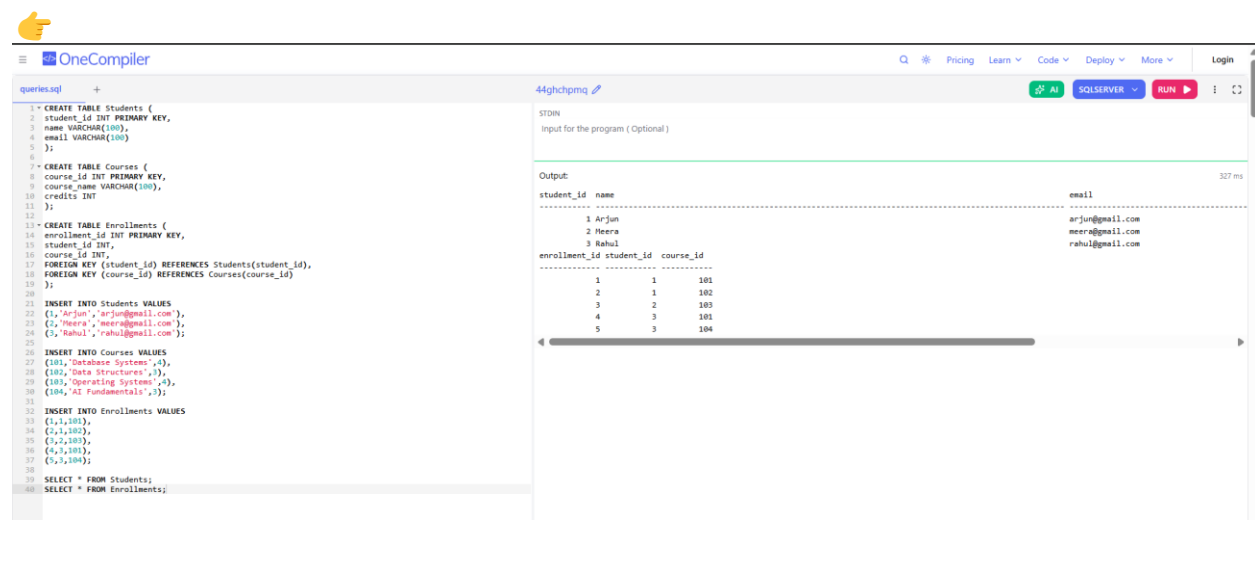
```
INSERT INTO Enrollments VALUES
(1, 1, 101),
(2, 1, 102),
(3, 2, 103),
(4, 3, 101),
(5, 3, 104);
```

```
SELECT * FROM Students;
SELECT * FROM Enrollments;
```

Output

Student and enrollment records displayed successfully.

Screenshot of Execution – Task 2



The screenshot shows the OneCompiler SQL editor interface. The left pane contains the SQL code for creating tables and inserting data. The right pane shows the output of the execution, which includes the data from the Students and Enrollments tables.

SQL Code:

```
1 CREATE TABLE Students (
2   student_id INT PRIMARY KEY,
3   name VARCHAR(100),
4   email VARCHAR(100)
5 );
6
7 CREATE TABLE Courses (
8   course_id INT PRIMARY KEY,
9   course_name VARCHAR(100),
10  credits INT
11 );
12
13 CREATE TABLE Enrollments (
14   enrollment_id INT PRIMARY KEY,
15   student_id INT,
16   course_id INT,
17   FOREIGN KEY (student_id) REFERENCES Students(student_id),
18   FOREIGN KEY (course_id) REFERENCES Courses(course_id)
19 );
20
21 INSERT INTO Students VALUES
22 (1, 'Arjun', 'arjun@gmail.com'),
23 (2, 'Meera', 'meera@gmail.com'),
24 (3, 'Rahul', 'rahul@gmail.com');
25
26 INSERT INTO Courses VALUES
27 (101, 'Database Systems', 4),
28 (102, 'Data Structures', 3),
29 (103, 'Operating Systems', 4),
30 (104, 'AI Fundamentals', 3);
31
32 INSERT INTO Enrollments VALUES
33 (1, 1, 101),
34 (2, 1, 102),
35 (3, 2, 103),
36 (4, 3, 101),
37 (5, 3, 104);
38
39 SELECT * FROM Students;
40 SELECT * FROM Enrollments;
```

Output:

student_id	name	email
1	Arjun	arjun@gmail.com
2	Meera	meera@gmail.com
3	Rahul	rahul@gmail.com

enrollment_id	student_id	course_id
1	1	101
2	1	102
3	2	103
4	3	101
5	3	104

Task 3 – Perform CRUD Operations

Prompt Used

Generate SQL queries for retrieving courses, updating student email and deleting an enrollment record.

SQL Code

```
SELECT * FROM Courses;
```

```
UPDATE Students
SET email='arjun_new@gmail.com'
WHERE student_id=1;
```

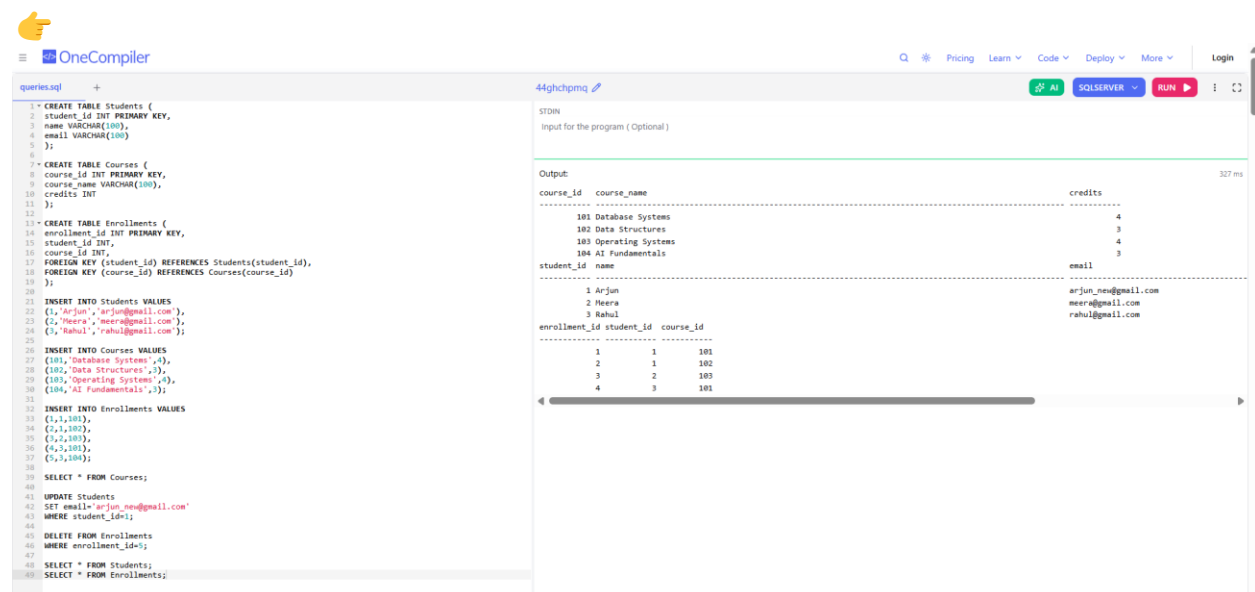
```
DELETE FROM Enrollments
WHERE enrollment_id=5;
```

```
SELECT * FROM Students;
SELECT * FROM Enrollments;
```

Output

Records updated and deleted successfully.

Screenshot of Execution – Task 3



The screenshot shows the OneCompiler interface with a SQL query editor on the left and a results pane on the right. The SQL code in the editor includes creating tables for Students, Courses, and Enrollments, inserting data, and performing update and delete operations. The results pane shows the output of the queries, including a list of courses and a list of students.

```
queries.sql
1 CREATE TABLE Students (
2   student_id INT PRIMARY KEY,
3   name VARCHAR(100),
4   email VARCHAR(100)
5 );
6
7 CREATE TABLE Courses (
8   course_id INT PRIMARY KEY,
9   course_name VARCHAR(100),
10  credits INT
11 );
12
13 CREATE TABLE Enrollments (
14   enrollment_id INT PRIMARY KEY,
15   student_id INT,
16   course_id INT,
17   FOREIGN KEY (student_id) REFERENCES Students(student_id),
18   FOREIGN KEY (course_id) REFERENCES Courses(course_id)
19 );
20
21 INSERT INTO Students VALUES
22 (1, 'Arjun', 'arjun@gmail.com'),
23 (2, 'Meera', 'meera@gmail.com'),
24 (3, 'Rahul', 'rahul@gmail.com');
25
26 INSERT INTO Courses VALUES
27 (101, 'Database Systems', 4),
28 (102, 'Data Structures', 3),
29 (103, 'Operating Systems', 4),
30 (104, 'AI Fundamentals', 3);
31
32 INSERT INTO Enrollments VALUES
33 (1, 1, 101),
34 (2, 1, 102),
35 (3, 2, 103),
36 (4, 3, 101),
37 (5, 3, 104);
38
39 SELECT * FROM Courses;
40
41 UPDATE Students
42 SET email='arjun_new@gmail.com'
43 WHERE student_id=1;
44
45 DELETE FROM Enrollments
46 WHERE enrollment_id=5;
47
48 SELECT * FROM Students;
49 SELECT * FROM Enrollments;
```

Output:

course_id	course_name	credits
101	Database Systems	4
102	Data Structures	3
103	Operating Systems	4
104	AI Fundamentals	3

student_id	name	email
1	Arjun	arjun_new@gmail.com
2	Meera	meera@gmail.com
3	Rahul	rahul@gmail.com

enrollment_id	student_id	course_id
1	1	101
2	1	102
3	2	103
4	3	101

Task 4 – Execute Aggregate Queries

Prompt Used

Generate SQL queries to count number of students per course and list courses having more than one student.

SQL Code

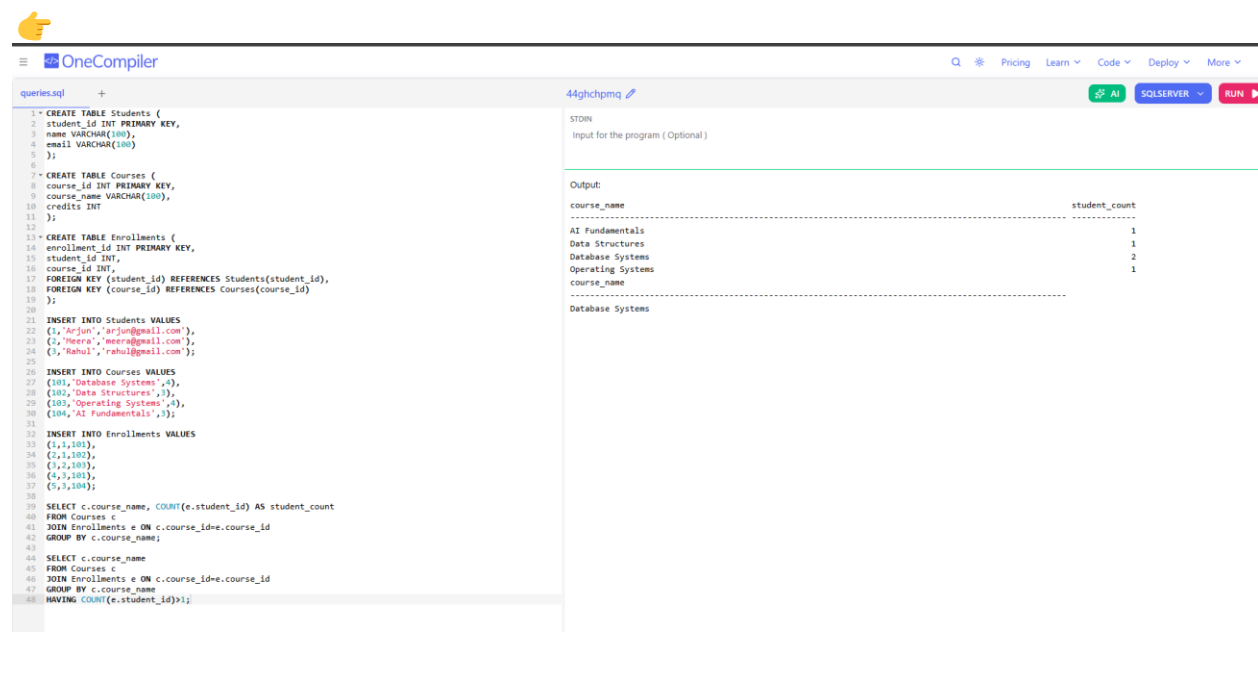
```
SELECT c.course_name, COUNT(e.student_id) AS student_count
FROM Courses c
JOIN Enrollments e ON c.course_id=e.course_id
GROUP BY c.course_name;
```

```
SELECT c.course_name
FROM Courses c
JOIN Enrollments e ON c.course_id=e.course_id
GROUP BY c.course_name
HAVING COUNT(e.student_id)>1;
```

Output

Student counts per course and courses with multiple enrollments displayed.

Screenshot of Execution – Task 4



The screenshot shows the OneCompiler interface. The left pane displays the following SQL code:

```
1 * CREATE TABLE Students (
2 student_id INT PRIMARY KEY,
3 name VARCHAR(100),
4 email VARCHAR(100)
5 );
6
7 * CREATE TABLE Courses (
8 course_id INT PRIMARY KEY,
9 course_name VARCHAR(100),
10 credits INT
11 );
12
13 * CREATE TABLE Enrollments (
14 enrollment_id INT PRIMARY KEY,
15 student_id INT,
16 course_id INT,
17 FOREIGN KEY (student_id) REFERENCES Students(student_id),
18 FOREIGN KEY (course_id) REFERENCES Courses(course_id)
19 );
20
21 INSERT INTO Students VALUES
22 (1, 'Arjun', 'arjun@gmail.com'),
23 (2, 'Meera', 'meera@gmail.com'),
24 (3, 'Rahul', 'rahul@gmail.com');
25
26 INSERT INTO Courses VALUES
27 (101, 'Database Systems', 4),
28 (102, 'Data Structures', 3),
29 (103, 'Operating Systems', 3),
30 (104, 'AI Fundamentals', 3);
31
32 INSERT INTO Enrollments VALUES
33 (1, 1, 101),
34 (2, 1, 102),
35 (3, 2, 103),
36 (4, 3, 101),
37 (5, 3, 104);
38
39 SELECT c.course_name, COUNT(e.student_id) AS student_count
40 FROM Courses c
41 JOIN Enrollments e ON c.course_id=e.course_id
42 GROUP BY c.course_name;
43
44 SELECT c.course_name
45 FROM Courses c
46 JOIN Enrollments e ON c.course_id=e.course_id
47 GROUP BY c.course_name
48 HAVING COUNT(e.student_id)>1;
```

The right pane shows the output of the first query:

course_name	student_count
AI Fundamentals	1
Data Structures	1
Database Systems	2
Operating Systems	1

Task 5 – Query Optimization

Prompt Used

Compare subquery and JOIN based query to retrieve students enrolled in course 101.

SQL Code

```
-- Original Query
SELECT * FROM Students
WHERE student_id IN (
SELECT student_id FROM Enrollments WHERE course_id=101
);
```

```
-- Optimized Query
SELECT s.*
FROM Students s
JOIN Enrollments e ON s.student_id=e.student_id
WHERE e.course_id=101;
```

Output

Both queries return the same students enrolled in course 101.

Screenshot of Execution – Task 5



OneCompiler

queries.sql +

44ghchpmq

```
1 CREATE TABLE Students (  
2   student_id INT PRIMARY KEY,  
3   name VARCHAR(100),  
4   email VARCHAR(100)  
5 );  
6  
7 CREATE TABLE Courses (  
8   course_id INT PRIMARY KEY,  
9   course_name VARCHAR(100),  
10  credits INT  
11 );  
12  
13 CREATE TABLE Enrollments (  
14   enrollment_id INT PRIMARY KEY,  
15   student_id INT,  
16   course_id INT,  
17   FOREIGN KEY (student_id) REFERENCES Students(student_id),  
18   FOREIGN KEY (course_id) REFERENCES Courses(course_id)  
19 );  
20  
21 INSERT INTO Students VALUES  
22 (1,'Arjun','arjun@gmail.com'),  
23 (2,'Meera','meera@gmail.com'),  
24 (3,'Rahul','rahul@gmail.com');  
25  
26 INSERT INTO Courses VALUES  
27 (101,'Database Systems',4),  
28 (102,'Data Structures',3),  
29 (103,'Operating Systems',4),  
30 (104,'AI Fundamentals',3);  
31  
32 INSERT INTO Enrollments VALUES  
33 (1,1,101),  
34 (2,1,102),  
35 (3,2,103),  
36 (4,3,101),  
37 (5,3,104);  
38  
39 SELECT * FROM Students  
40 WHERE student_id IN (  
41   SELECT student_id FROM Enrollments WHERE course_id=101  
42 );  
43  
44 SELECT s.*  
45 FROM Students s  
46 JOIN Enrollments e ON s.student_id=e.student_id  
47 WHERE e.course_id=101;
```

STDIN

Input for the program (Optional)

Output:

Msg 102, Level 15, State 1, Server 99a18edb38c1, Line 2
Incorrect syntax near '='.

Conclusion

This lab demonstrated relational database schema design, insertion of records, CRUD operations, aggregate queries and SQL query optimization using JOIN operations.